

Job Interview Prep Tool

github.com/Youcheng9/Job-Interview-Prep-Tool
Freeman Yiu (Coolguy4123) · Youcheng Taing (Youcheng9 Cheng)
Version 1.0 | March 2026 | 9-Week Development Sprint

Repository	github.com/Youcheng9/Job-Interview-Prep-Tool (main branch · 26 commits)
Contributors	Freeman Yiu — Coolguy4123 · Youcheng Taing — Youcheng9 Cheng
Languages	TypeScript 58.2% · Python 33.1% · CSS 7.0% · Other 1.7%
Repo Structure	backend/ · frontend/ · docs/ · .github/workflows/ · alembic.ini
Document Status	Draft v1.0 — Architecture Design for 9-Week Sprint

1. Introduction

1.1 Document Purpose

This Software Architecture Specification (SAS) provides a comprehensive architectural design for the Job Interview Prep Tool — a full-stack, AI-powered web application hosted at **github.com/Youcheng9/Job-Interview-Prep-Tool**. The platform enables structured, scored, and feedback-driven interview practice across four role tracks. This document describes each architectural layer, component responsibilities, data flows, external actors, and non-functional requirements to guide the 9-week development sprint.

1.2 Project Overview

The Job Interview Prep Tool is a full-stack AI-powered interview practice platform that helps users prepare for technical and behavioral interviews through structured questions, intelligent AI scoring, and personalised feedback. The repository is organised into two top-level application directories — **backend/** (Python / FastAPI) and **frontend/** (TypeScript / React) — with shared infrastructure configuration in **.github/workflows/**, database migration config in **alembic.ini**, and supplementary material in **docs/**.

1.3 Definitions & Acronyms

Term	Definition
SWE	Software Engineering — one of four supported interview role tracks
PM	Product Management interview track
DS	Data Science interview track
JWT	JSON Web Token — stateless authentication (15-min access / 7-day refresh)
ORM	Object-Relational Mapping — SQLAlchemy 2.0 database abstraction
RAG	Retrieval-Augmented Generation — exemplar-based contextual feedback (no external LLM)
SPA	Single-Page Application — the React/TypeScript frontend architecture
CI/CD	Continuous Integration/Deployment — GitHub Actions (.github/workflows/)
SAS	Software Architecture Specification — this document
MVP	Minimum Viable Product — target for 9-week sprint

2. Architectural Overview

2.1 Architectural Style

The platform adopts a **layered, service-oriented architecture** structured into five tiers: Presentation, Application API, ML/AI Pipeline, Data, and Infrastructure. Each tier maps to a distinct area of the repository and has well-defined boundaries. The frontend (TypeScript 58.2%) and backend (Python 33.1%) are developed independently and communicate exclusively over a versioned REST/HTTPS API, enabling the two-person team to work in parallel without coupling.

- Stateless FastAPI server enables horizontal scaling independently of the database and cache.
- ML pipeline runs in-process alongside FastAPI — no external AI API calls, no GPU required.
- React SPA consumes only the REST API; zero direct database access from the browser.
- Redis embedding cache (L3) decouples AI inference latency from repeated rubric lookups.
- Docker + docker-compose ensures environment parity between local development and Render production.
- GitHub Actions CI/CD (.github/workflows/) automates test, build, and deploy on every main-branch push.

2.2 Layer-to-Repository Mapping

Architecture Layer	Repo Directory	Language %	Responsibility
Presentation	frontend/	TypeScript 58.2%	React SPA, UI components, state, API calls, Recharts visualisation
Application API	backend/	Python 33.1%	FastAPI routes, business logic, JWT auth, Pydantic validation
ML / AI Pipeline	backend/ (internal)	Python 33.1%	Embedding, cosine scoring, keyword extraction, feedback generation
Data	backend/ + alembic.ini	Python 33.1%	PostgreSQL ORM (SQLAlchemy), Alembic migrations, Redis cache client
Infrastructure	.github/workflows/	YAML / Shell 1.7%	GitHub Actions CI/CD, Docker build, Render deploy hooks
Styles / Assets	frontend/ (CSS)	CSS 7.0%	Tailwind CSS utility classes, component-level styles

2.3 External Actors & Systems

Actor / System	Type	Interaction with Platform
End User (CS Student, Bootcamp Grad, Career Switcher, Early-Career Engineer)	Human Actor	Authenticates via browser SPA; selects role track; submits text answers; views AI scores, radar charts, and improvement feedback; tracks progress over time

University / Career Centre	Organisational Actor (future)	Future: org-level analytics dashboard, bulk student invitations — out of MVP scope
GitHub (Youcheng9/Job-Interview-Prep-Tool)	External System	Source code hosting (26 commits, main branch); triggers GitHub Actions CI/CD on push; stores container images in registry
Render Cloud Platform	External Hosting System	Hosts FastAPI Web Service container, React Static Site, managed PostgreSQL 15, managed Redis 7
Docker / Container Registry	External System	Stores Docker images built by GitHub Actions; images pulled by Render at deploy time
HuggingFace sentence-transformers (all-MiniLM-L6-v2)	External Library — runs locally	Model weights downloaded once during build/startup; all inference runs locally inside the backend/ container — zero runtime API calls or GPU required
Web Browser	Client Environment	Executes the React SPA; communicates with backend exclusively over HTTPS; handles JWT in-memory storage

3. Presentation Layer (frontend/)

The frontend lives in the **frontend/** directory and is a TypeScript-first React Single-Page Application — the largest share of the codebase at 58.2% TypeScript + 7.0% CSS. It consumes the FastAPI REST API exclusively and contains no business logic, ML code, or direct database access.

3.1 Components

Auth Module

Manages user login and registration flows. Stores JWT access token in memory (not localStorage) to mitigate XSS. Implements a protected route guard that validates the token before rendering any gated page. Communicates with `/api/v1/auth/register`, `/login`, and `/refresh`.

Role Selection Module

Presents the four available interview tracks — SWE, Data Science, PM, Behavioral — as selectable cards. On selection, calls the Questions API to load the role-specific question bank.

Q&A Interface

Renders interview questions one at a time with a text area for written answers. Handles submission to the Submission Service and routes the user to the Feedback Display on response.

Feedback Display

Renders AI scoring results: overall score (0–100), 4-axis radar chart (Technical Depth, Clarity, Completeness, Structure) via Recharts, listed strengths and weaknesses, and missing keywords/concepts surfaced by the Feedback Generator.

Score Dashboard

Aggregates all past submissions for the authenticated user. Displays per-question score history and overall improvement trend lines using Recharts line and radar chart components.

Answer History Page

Lists all previous submissions with timestamps, question text, and overall scores. Drill-down view shows the full AI feedback for any past answer — enabling users to track growth over time.

3.2 Key Libraries

Library	Version	Purpose
React	18.x	Component-based SPA framework
TypeScript	5.x	Type-safe JavaScript — primary frontend language (58.2%)
Tailwind CSS	3.x	Utility-first styling (7.0% of codebase)
Recharts	2.x	Radar charts, line charts, progress visualisation for scores

Axios / Fetch	—	HTTPS REST client; attaches JWT Bearer tokens to all API calls
---------------	---	--

4. Application Layer (backend/)

The backend lives in the **backend/** directory and is implemented with FastAPI (Python 3.11+), part of the 33.1% Python codebase. It exposes a versioned REST API under **/api/v1/**. All incoming request bodies are validated by Pydantic models before any business logic or database access occurs.

4.1 Modules

API Gateway / Request Router

Central entry point for all client requests. Enforces CORS policy, attaches JWT Auth Middleware via FastAPI dependency injection, applies rate limiting, and dispatches to downstream modules. All Pydantic schema validation happens at this boundary.

Auth Service

Handles user registration and login. Hashes passwords with bcrypt (cost factor 12) before storage. Issues JWT access tokens (15 min) and refresh tokens (7 days). Token refresh endpoint issues new access tokens without requiring re-login.

Questions API

Serves paginated, role-filtered questions from the PostgreSQL question bank. Each response includes the question text, difficulty, rubric components, and keyword list. Supports optional randomisation for varied practice sessions.

Submission Service

Receives user answer text, delegates scoring to the ML/AI Pipeline layer, persists the submission record, multi-dimensional score, and all feedback items to PostgreSQL, then returns structured score JSON to the frontend in a single response.

Progress API

Aggregates submission history per authenticated user. Calculates per-question score trends and overall improvement metrics across all attempts. Powers the Score Dashboard and Answer History Page.

User Profile Service

Manages account details and user preferences. Provides GET and PATCH endpoints. Future scope: notification settings, preferred role track, display preferences.

4.2 API Endpoint Reference

Method	Endpoint	Description	Auth
POST	/api/v1/auth/register	Register new user account	No
POST	/api/v1/auth/login	Authenticate; receive JWT access + refresh tokens	No

POST	/api/v1/auth/refresh	Issue new access token from refresh token	Refresh Token
GET	/api/v1/questions	List questions filtered by role; paginated	Yes
GET	/api/v1/questions/{id}	Single question with rubric components + keywords	Yes
POST	/api/v1/submissions	Submit answer; returns 4-axis score + feedback	Yes
GET	/api/v1/submissions	Authenticated user's full submission history	Yes
GET	/api/v1/submissions/{id}	Single scored submission with full feedback	Yes
GET	/api/v1/progress	Aggregated score trends and improvement metrics	Yes
GET	/api/v1/users/me	Get authenticated user profile	Yes
PATCH	/api/v1/users/me	Update profile / preferences	Yes

5. ML / AI Pipeline (backend/ — internal modules)

The ML pipeline is implemented as internal Python modules co-located inside **backend/** alongside the FastAPI application. It requires no external AI API calls, no fine-tuning, and no GPU — all inference runs locally using the `sentence-transformers` library (all-MiniLM-L6-v2, 384-dimensional embeddings). This architectural choice eliminates per-request AI costs and keeps scoring deterministic and auditable.

5.1 Components

Answer Embedder

Converts user answer text and rubric component strings into 384-dimensional float32 dense vectors using the all-MiniLM-L6-v2 model. Rubric and exemplar embeddings are looked up in Redis first (SHA-256 keyed, 7-day TTL); on a cache miss the model runs and the result is stored. User answer embeddings are always computed fresh (not cached) since they are unique per submission.

Cosine Similarity Scorer

Computes cosine similarity between the user answer embedding and each rubric component embedding. Produces a per-element coverage score in [0.0, 1.0]. Applies configurable per-component weights and aggregates into an overall rubric coverage score for the submission.

Keyword Coverage Extractor

Performs lightweight string matching of the user answer against the per-question keyword list. Keywords carry a required/optional flag: missing required keywords are surfaced as high-priority feedback items; missing optional keywords are surfaced as enhancement suggestions. Identified missing keywords feed directly into the Feedback Generator output.

Multi-Dimensional Score Engine

Combines outputs from the Cosine Similarity Scorer and Keyword Coverage Extractor to produce four independent dimension scores (0–100 scale):

Technical Depth — coverage of domain-specific concepts and required keywords.

Clarity — answer coherence and semantic alignment with model rubric phrasing.

Completeness — ratio of rubric components addressed at or above the threshold.

Structure — presence of logical flow indicators (STAR format for behavioral, step-by-step for technical).

Feedback Generator

Receives dimension scores and rubric coverage gaps from the Score Engine. Formats the results into a structured JSON payload containing: overall score (0–100), the 4-axis breakdown, strengths (high-coverage rubric elements), weaknesses (low-coverage elements), and missing keywords/concepts. This is the final AI output consumed by the Submission Service and rendered in the Feedback Display component.

RAG Pipeline (Retrieval-Augmented Generation)

Maintains a pre-embedded vector store of exemplar answers and rubric descriptions. At submission time, retrieves the semantically closest exemplar answers via cosine similarity to provide contextual reference in the feedback payload. Implemented entirely with in-process cosine retrieval — no external LLM API is required.

5.2 Scoring Data Flow

```
Answer text → Answer Embedder → [Redis cache check] → Cosine Similarity Scorer + Keyword  
Coverage Extractor → Multi-Dimensional Score Engine → Feedback Generator → Structured  
JSON {overall, tech_depth, clarity, completeness, structure, strengths, weaknesses,  
missing_keywords} → Submission Service → PostgreSQL + Frontend
```

6. Data Layer (backend/ + alembic.ini)

6.1 PostgreSQL 15 — Primary Persistent Store

All application state is persisted in a Render-managed PostgreSQL 15 instance. Schema evolution is managed by **Alembic** (configured in the repository root **alembic.ini**), with migration scripts versioned in **backend/**. All SQL access goes through SQLAlchemy 2.0 ORM — no raw SQL strings in application code.

6.2 Database Schema

Table	Key Columns	Purpose
users	id (UUID PK), email (unique), password_hash, created_at, preferences (JSONB)	User accounts, bcrypt-hashed credentials, and user profile preferences
questions	id (UUID PK), role ENUM('SWE','DS','PM','Behavioral'), title, difficulty, created_at	Role-tagged interview question bank; role ENUM enforces valid track values
rubric_components	id, question_id (FK), component_text, weight FLOAT, category	Weighted scoring rubric elements per question; weight controls score contribution
keywords	id, question_id (FK), keyword, is_required BOOL	Domain-specific terms for keyword coverage scoring; required flag = high-priority feedback
submissions	id, user_id (FK), question_id (FK), answer_text TEXT, submitted_at	Raw user answer submissions — append-only to preserve full history
scores	id, submission_id (FK), technical_depth, clarity, completeness, structure, overall (all INT 0-100)	4-axis + overall score record per submission
feedback_items	id, submission_id (FK), rubric_component_id (FK), suggestion_text, covered BOOL	Per-rubric-element AI feedback: suggestion text and whether the element was covered

6.3 Redis 7 — Embedding Cache (L3)

A Render-managed Redis 7 instance caches pre-computed float32[384] embeddings for rubric components and RAG exemplar answers. Cache keys are SHA-256 hashes of the source text, ensuring exact-match invalidation. TTL is 7 days. On cache miss, the Answer Embedder recomputes and stores the result. This eliminates redundant ML inference for stable rubric content and keeps the scoring API response time under 3 seconds at p95.

7. Infrastructure & Deployment (.github/workflows/)

7.1 Containerisation

Both the FastAPI + ML backend and the React frontend are containerised with Docker. A **docker-compose** configuration at the repository root enables single-command local development, spinning up: the FastAPI API server (with ML pipeline), the React dev server, a local PostgreSQL 15 container, and a local Redis 7 container. This mirrors the Render production topology exactly.

7.2 Render Production Deployment

- **Web Service** — FastAPI + ML pipeline Docker container. Auto-scales horizontally. TLS/HTTPS enforced by Render.
- **Static Site** — React production build. Served via Render's CDN with global edge delivery and HTTPS.
- **Managed PostgreSQL 15** — Render-hosted; automatic daily backups; Alembic migrations run post-deploy.
- **Managed Redis 7** — Render-hosted embedding cache; persistent across deploys.

7.3 GitHub Actions CI/CD (.github/workflows/)

Every push to the **main** branch triggers the CI/CD pipeline defined in **.github/workflows/**:

- Run Python test suite (**pytest**) against the backend/ codebase.
- Run TypeScript type checks (**tsc --noEmit**) against the frontend/ codebase.
- Build Docker image for the FastAPI/ML service and push to the container registry.
- Build the React production bundle.
- Trigger Render deploy hooks for both the Web Service and Static Site.
- Run Alembic migrations against the production PostgreSQL database on successful deploy.

7.4 Environment Matrix

Environment	Trigger	Services	Notes
Local Dev	docker-compose up	FastAPI + ML · React dev server · PostgreSQL (local) · Redis (local)	Hot-reload on both frontend and backend; mirrors production config via .env files
Production	Push to main → GitHub Actions	Render Web Service · Render Static Site · Render PostgreSQL · Render Redis	Auto-deployed; Alembic migrations run post-deploy; TLS enforced

8. Security Architecture

8.1 Authentication & Authorisation

- JWT-based stateless authentication. Access tokens expire in 15 minutes; refresh tokens expire in 7 days.
- Passwords hashed with **bcrypt** (cost factor 12) before storage — plaintext never persisted.
- All API endpoints except **/auth/register** and **/auth/login** require a valid Bearer token validated via FastAPI dependency injection middleware.
- Row-level authorisation enforced at the service layer: users may only read and modify their own submissions, scores, and feedback items.
- JWT stored **in-memory only** on the frontend (never in localStorage or sessionStorage) to eliminate XSS token theft.

8.2 Transport & HTTP Security

- All traffic uses HTTPS/TLS — enforced by Render (no plain HTTP reachable in production).
- CORS policy restricts allowed origins to the registered Render static-site domain.
- HTTP security headers: HSTS, X-Content-Type-Options, X-Frame-Options set at API level.

8.3 Input Validation & Injection Prevention

- All request bodies validated by Pydantic models in FastAPI before any business logic or DB access.
- Answer text capped at 10,000 characters to prevent ML pipeline abuse.
- SQL injection prevented by exclusive use of SQLAlchemy ORM with parameterised queries — no raw SQL strings in application code.
- Alembic migration scripts versioned in the repository and reviewed before production deployment.

9. Non-Functional Requirements

Category	Requirement	Target / Measure
Performance	Scoring API response time (p95)	< 3 seconds (Redis cache eliminates repeated embedding cost)
Performance	Frontend initial load — Largest Contentful Paint	< 2 seconds on a 4G connection
Availability	Production uptime (Render managed SLA)	> 99%
Scalability	Concurrent users — MVP target	Up to 500 concurrent users on Render Web Service
Maintainability	Python unit test coverage (pytest)	> 70% for ML pipeline modules in backend/
Portability	Local dev environment setup time	< 5 minutes with docker-compose up
Security	JWT token expiry	15-min access token / 7-day refresh token
Security	Answer text input size limit	10,000 characters maximum per submission
Data Retention	Submission history	Indefinite — user-owned; no automatic deletion policy
Observability	Error and request logging	Structured JSON logs captured by Render log drain
Reproducibility	ML scoring	Deterministic — no stochastic sampling; same input always yields same score

10. Development Timeline

Phase	Weeks	Owner	Deliverables (repo paths)
Backend + DB	1–3	Youcheng Taing (Youcheng9)	backend/: FastAPI project setup, PostgreSQL schema, Alembic migrations (alembic.ini), JWT auth service, Questions API, Docker container, initial .github/workflows/ CI skeleton
ML Pipeline	4–5	Both	backend/ (ML modules): Answer Embedder (all-MiniLM-L6-v2), Cosine Similarity Scorer, Keyword Coverage Extractor, Multi-Dim Score Engine, Feedback Generator, RAG Pipeline, Redis cache integration
Frontend	6–7	Freeman Yiu (Coolguy4123)	frontend/: React SPA, Role Selection UI, Q&A; Interface, Feedback Display (Recharts radar), Score Dashboard (trend lines), Answer History Page, TypeScript types, Tailwind CSS styles
Integration + Deploy	8–9	Both	docker-compose full-stack compose, Render deployment (Web Service + Static Site + managed DB + Redis), .github/workflows/ full CI/CD pipeline, QA testing, docs/ updates, demo preparation

11. Appendix — Technology Stack

Technology	Version	Repo Dir	Role in System
TypeScript	5.x	frontend/	Primary frontend language — 58.2% of codebase
React	18.x	frontend/	SPA component framework
Tailwind CSS	3.x	frontend/	Utility-first UI styling — 7.0% of codebase (CSS)
Recharts	2.x	frontend/	Radar chart + line chart score visualisation
Python	3.11+	backend/	Backend + ML runtime — 33.1% of codebase
FastAPI	0.110+	backend/	Async REST API framework with Pydantic validation
SQLAlchemy	2.0+	backend/	ORM — all database access
Alembic	1.13+	alembic.ini + backend/	PostgreSQL schema migration manager
bcrypt	4.x	backend/	Password hashing (cost factor 12)
sentence-transformers	2.x	backend/	Local ML embedding library
all-MiniLM-L6-v2	HuggingFace	backend/	384-dim embedding model — runs locally, no GPU needed
PostgreSQL	15+	Render managed	Primary persistent data store
Redis	7.x	Render managed	Embedding cache (L3) — SHA-256 keys, 7-day TTL
Docker	24+	repo root	Containerisation for both services
docker-compose	2.x	repo root	Local dev multi-service orchestration
Render	Managed Cloud	—	Hosting: Web Service, Static Site, PostgreSQL, Redis
GitHub Actions	—	.github/workflows/	CI/CD — test, build, push, deploy, migrate